

Extensions on older versions

You can force some of the extensions you write using the Add-On SDK to work on older versions of Firefox

Electrolysis project

Mozilla has a project called Electrolysis which aims to move Firefox to the Google Chrome model of different processes for each tab

Creating your own Firefox add-on using the new SDK

Kshitij Sobti

kshitij.sobti@thinkdigit.com

Firefox is an incredibly extensible application, not only because there are a large number of add-ons available for the browser, but also because the add-ons for Firefox can make much deeper and more extensive changes to it.

Its extensibility emerges from the way the browser is built. Firefox's GUI is not coded in C/C++, but is instead written in an XML dialect called XUL (XML User Interface Language). A large part of Firefox's functionality is written in JavaScript, and the browser is styled using an extended form of CSS.

This XUL + CSS + JavaScript code runs on a framework called XULRunner that renders the XUL code just as a browser would run HTML code. The browser is then running on the same JavaScript engine which web applications will be running on. At its very core XULRunner is a collection of cross-platform components that includes an XUL and HTML renderer, a networking library, a SQLite database etc. These components can be used by the application running on XULRunner to provide functionality. In fact Firefox, Thunderbird, and Songbird, three very different applications, all run on Firefox!

Applications that extend Firefox or any XULRunner-based application overlay their XUL code over that of the application, and are thus able to modify any part of the application's UI and functionality.

Unfortunately, developing add-ons or applications is not as simple as it might seem. XULRunner unlike other runtimes such as Adobe AIR, and Titanium was not designed to build applications on, and is thus not that well documented. There are few IDEs for developing XULRunner applications. To make matters worse, changes made to Firefox in between ver-



Firefox 4 promises to be even easier for Add-On developers to dive into, thanks to the new Firefox Add-On SDK. In this article you will take your first steps in developing a simple add-on to post to Twitter.

sions can stop add-ons written for an older version from working in a newer version.

Now let's move on to how you can use the Add-On SDK to build extensions for Firefox 4.0 today. We will be creating an extension that will modify the Firefox context menu by adding an option to tweet the text you have selected. This won't use the proper API for Twitter since that will involve authentication which will be out of the scope of this article. Instead we will use a simpler system that will open Twitter with the text to tweet entered in the tweet box.

Getting started with the Add-On SDK

The first step of course is to get the Add-On SDK itself and extract it to a convenient place (the SDK works on Windows Linux and Mac OSX). You will also need Python installed to use the SDK. Install it now from the Digit DVD if you haven't already.

Since the Add-On SDK is made up of command line tools, you should launch the terminal in your OS. Now you will need to move to the directory in which you have the Add-On SDK installed ("C:\addonsdk" for example) and type "bin\activate" ("source bin/activate" on Linux) to run the activate command which will take you into the Add-On SDK environment.

The command you will be using most of all will be 'cfx'. The cfx tool will auto-

matically find and use any Firefox instance installed for testing extensions. It might launch Firefox 3.6 if you have that installed along with Firefox 4, so it's better to specify the particular version of Firefox to use in the command-line. How exactly, we will cover in a bit.

At this point, it helps to run 'cfx docs' from the command line. This will have the Add-On SDK launch a local server for the documentation, and will open the same in a browser.

Create a directory for your add-on (we'll assume you named it "myaddon"), and enter it in the Add-On SDK environment, then type "cfx init" to initialise the basic structure of the add-on in that directory.

The manifest

Before we start with the actual code for the extension, let us first look at the manifest that will contain the meta-data (extension name, author name, version, license, description etc.).

Inside the folder you created for your add-on, you will find a file called 'package.json'. This is a plain text file that should look something like this:

Here is a sample "package.json" file:

```
{
  "name": "myaddon",
  "fullName": "myaddon",
  "description": "a basic add-on",
  "author": "",
  "license": "MPL 1.1/GPL 2.0/LGPL 2.1",
  "version": "0.1"
}
```

The contents of this file are self-explanatory, however do take note of the format, which is something called JSON (JavaScript Object Notation). These are just the basic metadata parameters for the file, there are many others. Not all parameters are necessary, and there is one important parameter "id" that is missing, but will be generated later.

We will now begin with the actual coding for the extension.

The main module

The Add-On SDK is very modular, and uses a popular module standard called CommonJS. Any modules you